

Programowanie obiektowe

Bartosz Miąskowski

Enkapsulacja / Hermetyzacja

Enkapsulacja (*encapsulation*, zwana też *hermetyzacją*) to zasada programowania obiektowego polegająca na **ukrywaniu szczegółów implementacji** wewnątrz klasy i **udostępnianiu tylko niezbędnego interfejsu** zewnętrznego.

Cele:

- Ochrona danych przed niepożądanymi zmianami.
- Zwiększenie bezpieczeństwa i kontroli dostępu.
- Ułatwienie utrzymania kodu (zmiana wewnętrzna nie wymaga zmian w kodzie zewnętrznym).
- Poprawa czytelności i modularności programu

Enkapsulacja / Hermetyzacja

Analogia – samochód

Wyobraź sobie, że obiekt to samochód.

- Jako kierowca wchodzisz w interakcję z samochodem za pomocą prostego i dobrze zdefiniowanego **interfejsu**: kierownicy, pedałów, dźwigni zmiany biegów i deski rozdzielczej.
- Nie musisz wiedzieć, jak dokładnie działa silnik spalinowy, jak skonstruowany jest układ hamulcowy ani jak złożona elektronika zarządza pracą poszczególnych komponentów.
- Cała ta wewnętrzna złożoność jest **ukryta "pod maską"**. Jest celowo "zamknięta", aby chronić mechanizmy przed przypadkową ingerencją i jednocześnie uprościć obsługę.

Enkapsulacja w kodzie

W programowaniu obiektowym tę "kapsułkę" tworzy klasa.

- Łączy ona w sobie pola (dane) oraz metody (funkcje), które na tych danych operują.
- Poprzez użycie modyfikatorów dostępu, takich jak **public** i **private**, klasa decyduje, które jej elementy są częścią publicznego interfejsu (dostępne z zewnątrz), a które są prywatnymi szczegółami implementacji.
- Głównym celem jest ochrona wewnętrznego stanu obiektu i zapewnienie jego spójności. Zmiany w stanie obiektu mogą zachodzić tylko poprzez jego publiczne metody, co daje pełną kontrolę nad danymi.

Enkapsulacja / Hermetyzacja

```
class Account
{
    // Pole prywatne – ukryte przed światem zewnętrznym
    private decimal _balance;

    // Właściwość publiczna – kontrolowany dostęp do pola
    public decimal Balance
    {
        get => _balance;
        private set
        {
            if (value >= 0)
                _balance = value;
        }
    }

    public void Deposit(decimal amount)
    {
        if (amount > 0)
            Balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount > 0 && Balance >= amount)
            Balance -= amount;
    }
}
```

- Dane są **ukryte** (np. private _balance).
- Dostęp kontrolowany przez **właściwości i metody**.
- Kod zewnętrzny nie może zmienić stanu obiektu w niekontrolowany sposób

Enkapsulacja / Hermetyzacja

Dlaczego ukrywanie szczegółów implementacji jest ważne?

Pomyśl, co by się stało, gdyby wszystkie pola w Twoich klasach były **publiczne**. Każdy fragment kodu mógłby je dowolnie modyfikować, bez żadnej kontroli. Prowadziłoby to do chaosu i trudnych do znalezienia błędów. Ukrywanie implementacji (zasada *information hiding*) jest fundamentem stabilnego i łatwego w utrzymaniu oprogramowania.

Enkapsulacja / Hermetyzacja

Główne powody, dla których stosujemy hermetyzację, to:

- **Kontrola i spójność danych (integralność)**

Dzięki ukryciu pól i udostępnieniu metod (lub właściwości) do ich modyfikacji, masz pełną kontrolę nad stanem obiektu. Możesz dodać logikę walidacyjną, aby nie dopuścić do przypisania nieprawidłowych wartości.

Przykład: Właściwość Wiek może sprawdzać, czy przypisywana wartość nie jest ujemna. Bezpośredni dostęp do publicznego pola `public int wiek`; uniemożliwiłby taką kontrolę.

- **Uproszczenie interfejsu (łatwość użycia)**

Użytkownik Twojej klasy nie musi wiedzieć, *jak* ona działa w środku. Interesuje go tylko, co potrafi zrobić. Dostaje prosty zestaw publicznych metod, które są jego narzędziami, a cała złożoność pozostaje ukryta.

- **Elastyczność i łatwość w utrzymaniu (refaktoryzacja / rozwój)**

To jedna z największych zalet. Możesz całkowicie zmienić wewnętrzną implementację klasy – np. sposób przechowywania danych, użyte algorytmy – a dopóki jej publiczny interfejs pozostaje taki sam, kod, który z niej korzysta, nie ulegnie zniszczeniu i nie będzie wymagał zmian.

Enkapsulacja / Hermetyzacja

Kiedy nie używamy hermetyzacji... 🤔

```
class Account
{
    // Wszystko publiczne – każdy może zrobić, co chce 🤖
    public decimal Balance;
}

class Program
{
    static void Main()
    {
        var acc = new Account();
        acc.Balance = -1000; // ❌ niekontrolowana zmiana!
        Console.WriteLine($"Saldo: {acc.Balance} zł");
    }
}
```

Kod zewnętrzny może dowolnie manipulować danymi, wprowadzać błędy, a klasa traci kontrolę nad swoim stanem.

Elegancko! Z hermetyzacją. 😊

```
class Account
{
    private decimal _balance;

    public decimal Balance => _balance;

    public void Deposit(decimal amount)
    {
        if (amount > 0)
            _balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount > 0 && amount <= _balance)
            _balance -= amount;
    }
}

class Program
{
    static void Main()
    {
        var acc = new Account();
        acc.Deposit(500);
        acc.Withdraw(200);
        Console.WriteLine($"Saldo: {acc.Balance} zł");
    }
}
```

Tylko metody Deposit() i Withdraw() mogą zmienić saldo. Programista korzystający z klasy **nie musi wiedzieć, jak działa** logika wewnętrzna – wystarczy, że zna interfejs publiczny.

Abstrakcja

Ukrywanie implementacji to forma abstrakcji

- **Abstrakcja** w programowaniu to proces ukrywania złożonych szczegółów implementacyjnych i pokazywania użytkownikowi (innemu programiście) tylko niezbędnych, uproszczonych funkcjonalności. Chodzi o skupienie się na tym, **co** obiekt robi, a nie na tym, **jak** to robi.
- Enkapsulacja jest więc praktyczną realizacją zasady abstrakcji na poziomie obiektu. Tworząc klasę, tworzysz nowy "typ abstrakcyjny" – koncept, który ma określone zachowania (metody publiczne), ale którego wewnętrzna maszyna jest ukryta przed światem.

Klasy wewnętrzne a hermetyzacja

Główna zasada jest prosta – klasa wewnętrzna jest traktowana jako składowa klasy zewnętrznej. Dzięki temu, nawet jeśli jest zadeklarowana jako `private`, może uzyskiwać dostęp do **prywatnych** składowych (pól, metod) klasy zewnętrznej.

Dlaczego to jest tak ważne dla enkapsulacji?

- **Ukrywanie klas pomocniczych** – czasami potrzebujesz klasy, która służy wyłącznie jako "pomocnik" dla innej, głównej klasy. Nie ma żadnego powodu, aby taka klasa była widoczna dla reszty programu. Umieszczenie jej wewnątrz klasy głównej i oznaczenie jako `private` sprawia, że staje się ona niewidocznym szczegółem implementacyjnym. Nikt spoza klasy zewnętrznej nie może nawet utworzyć jej instancji.
- **Logiczne grupowanie** – jeśli jakaś klasa jest nierozzerwalnie związana z inną i nie ma sensu bytowego bez niej, umieszczenie jej wewnątrz jest naturalnym sposobem na zorganizowanie kodu. To sygnał dla innych programistów: "ta klasa jest wewnętrznym mechanizmem i nie powinna być używana bezpośrednio".
- **Uproszczony dostęp do danych** – klasa pomocnicza, zamiast komunikować się z klasą główną przez jej publiczny interfejs, może bezpośrednio (i bezpiecznie) operować na jej prywatnych polach, ponieważ jest jej "zaufaną" częścią.

Klasy wewnętrzne a hermetyzacja

Co tu się dzieje?

- PasswordManager to klasa pomocnicza **niewidoczna na zewnątrz**
- Logika związana z hasłami jest **ukryta przed innymi klasami**
- Klasa User udostępnia tylko **czysty interfejs publiczny** – VerifyPassword()

Wniosek:

Klasy wewnętrzne pozwalają **zamknąć** szczegóły implementacji w ramach jednej jednostki logicznej, dzięki czemu **wnętrze klasy pozostaje prywatne**, a świat zewnętrzny widzi tylko to, co istotne.

```
class User
{
    private string _name;
    private PasswordManager _passwordManager; // klasa pomocnicza

    public User(string name, string password)
    {
        _name = name;
        _passwordManager = new PasswordManager(password);
    }

    public bool VerifyPassword(string input)
        => _passwordManager.Check(input);

    // Klasa wewnętrzna – widoczna tylko w obrębie User
    private class PasswordManager
    {
        private string _hashedPassword;

        public PasswordManager(string plainPassword)
        {
            _hashedPassword = Hash(plainPassword);
        }

        private string Hash(string input)
            => $"{input.GetHashCode()}"; // uproszczony przykład

        public bool Check(string input)
            => _hashedPassword == Hash(input);
    }
}
```

Interfejsy

Definicja:

Interfejs to umowa (kontrakt), która określa jakie metody i właściwości klasa musi posiadać, ale nie mówi, jak mają być zaimplementowane.

Co to oznacza?

- Interfejs **nie zawiera implementacji**, tylko **sygnatury** metod/właściwości.
- Klasa, która **implementuje interfejs**, musi dostarczyć konkretną realizację każdej z nich.
- Interfejsy umożliwiają **wspólny sposób komunikacji** między różnymi klasami.
- To przykład **abstrakcji** – skupiamy się na tym, *co obiekt potrafi, a nie jak to robi*.

Kluczowa zasada – interfejs określa **CO** obiekt musi potrafić zrobić, ale absolutnie nie mówi, **JAK** ma to zostać wykonane.

Interfejsy

```
// Definicja interfejsu
interface IPlayable
{
    void Play();
    void Pause();
}

// Implementacja interfejsu przez klasę
class MusicPlayer : IPlayable
{
    public void Play() => Console.WriteLine("🎵 Odtwarzanie muzyki...");
    public void Pause() => Console.WriteLine("⏸ Pauza");
}

// Inna klasa implementująca ten sam interfejs
class VideoPlayer : IPlayable
{
    public void Play() => Console.WriteLine("▶ Odtwarzanie filmu...");
    public void Pause() => Console.WriteLine("⏸ Pauza filmu");
}
```

MusicPlayer i VideoPlayer **robią różne rzeczy**, ale można je **traktować tak samo**, bo oba implementują IPlayable.

```
IPlayable player = new MusicPlayer();
player.Play(); // "🎵 Odtwarzanie muzyki..."
```

Wniosek:

Interfejs to obietnica zachowania – mówi co obiekt potrafi, ale nie zdradza jak to robi. Dzięki temu kod jest elastyczny i łatwy w rozbudowie.

Interfejsy

Wyobraź sobie, że zlecasz firmie budowlanej budowę domu. Podpisujecie umowę (interfejs), która precyzuje:

- Dom musi mieć możliwość otwierania i zamykania drzwi wejściowych.
- Musi mieć możliwość włączania i wyłączania światła w każdym pokoju.
- Musi zapewniać możliwość odkręcania i zakręcania wody w kuchni.

Umowa ta **nie określa**, jakiego rodzaju zamków użyć, jakiej firmy żarówek, ani jakiej technologii rur hydraulicznych. Jedna firma (klasa `FirmaBudowlanaA`) może użyć zamków cyfrowych i oświetlenia LED, a inna (klasa `FirmaBudowlanaB`) tradycyjnych zamków na klucz i zwykłych żarówek.

Dla Ciebie, jako klienta, obie firmy spełniają warunki kontraktu. Możesz wejść w interakcję z każdym z tych domów w ten sam, zdefiniowany sposób (otworzyć drzwi, włączyć światło), nie martwiąc się o szczegóły implementacyjne.

Interfejsy

Kluczowe zasady tworzenia interfejsów

- **Tylko deklaracje, nie implementacje:** W swoim podstawowym założeniu interfejs zawiera wyłącznie sygnatury (nagłówki) metod, właściwości, zdarzeń i indeksatorów. Nie ma w nim logiki (ciał metod).
- **Wszystkie składowe są domyślnie publiczne:** Nie musisz (i w starszych wersjach C# nie mogłeś) pisać modyfikatora public przed składowymi interfejsu. Każdy element kontraktu jest z definicji publiczny, ponieważ musi być widoczny i możliwy do zaimplementowania przez klasę.
- **Brak pól i konstruktorów:** Interfejs nie może zawierać pól (zmiennych instancyjnych), ponieważ nie przechowuje stanu. Definiuje on wyłącznie zachowanie. Z tego samego powodu nie ma konstruktorów – interfejsu nie można utworzyć bezpośrednio za pomocą słowa kluczowego new.

```
// Domyślnie interfejsy mają dostęp publiczny
public interface IMyInterface
{
    // 1. Deklaracja metody (bez ciała, tylko sygnatura)
    void DoSomething();

    // 2. Deklaracja właściwości (bez implementacji logiki get/set)
    string Name { get; set; }

    // 3. Deklaracja zdarzenia
    event Action StateChanged;

    // 4. Deklaracja indeksatora
    int this[int index] { get; set; }
}
```

Interfejsy

```
public interface IDrawable
{
    // Każdy obiekt, który można narysować, musi mieć kolor.
    // Właściwość może być tylko do odczytu (get) lub do zapisu (set).
    ConsoleColor Color { get; set; }

    // Każdy taki obiekt musi udostępniać metodę Draw.
    void Draw();
}
```

Stwórzmy prosty interfejs dla obiektów, które można narysować na ekranie. Chcemy, aby każdy taki obiekt miał kolor i metodę do rysowania.

Każda klasa, która zaimplementuje ten interfejs (np. Kolo, Prostokat, Gwiazdka), będzie musiała dostarczyć publiczną właściwość Color oraz publiczną metodę Draw. Dzięki temu zyskujemy pewność, że każdy obiekt typu IDrawable na pewno potrafi siebie narysować.

Interfejsy – zasady projektowania

Małe i spójne (Zasada Segregacji Interfejsów)

Jeden duży interfejs, który robi wszystko, to zły pomysł. Zamiast tego, twórz wiele małych, wyspecjalizowanych interfejsów, z których każdy opisuje jedną, konkretną zdolność. Klasa nie powinna być zmuszana do implementowania metod, których nie potrzebuje.

- **Źle:** Jeden wielki interfejs IPojazd z metodami Jedz(), Lataj(), Plywaj(). Samochód musiałby implementować Lataj() i Plywaj(), nawet jeśli nic w nich nie robi.
- **Dobrze:** Trzy małe interfejsy: IJedzacy, ILatajacy, IPlywajacy. Klasa Samolot zaimplementuje IJedzacy (kołowanie po lotnisku) i ILatajacy. Klasa Amfibia zaimplementuje IJedzacy i IPlywajacy.

Interfejsy – zasady projektowania

Nazwa powinna jasno komunikować, co dany interfejs reprezentuje.

- Używaj przymiotników zakończonych na -able (lub ich polskich odpowiedników), np. `IDisposable` (zdolny do zwolnienia zasobów), `IEnumerable` (zdolny do wyliczania), `IComparable` (zdolny do porównania).
- Używaj rzeczowników opisujących rolę lub zachowanie, np. `ILogger` (rejestrator zdarzeń), `IValidator` (walidator), `IMessagePublisher` (wydawca wiadomości).
- Pamiętaj o konwencji prefiksu `I`.

Interfejsy – zasady projektowania

Projektuj z perspektywy klienta

- Zawsze myśl o tym, **jak ten interfejs będzie używany**. Skup się na potrzebach kodu, który będzie z niego korzystał ("klienta"). Interfejs ma upraszczać życie programisty, a nie je komplikować. Zadać sobie pytanie: "Jakiego minimalnego zestawu metod potrzebuje klient, aby wykonać swoje zadanie?". Wszelkie szczegóły implementacyjne powinny być ukryte.

Interfejsy – zasady projektowania

Skupiaj się na "CO", a nie na "JAK"

Nazwy metod w interfejsie powinny opisywać intencję, a nie konkretny sposób jej realizacji. Daje to swobodę klasom implementującym.

- Dobrze: `ZapiszDane()` zamiast `ZapiszDaneDoPlikuJSON()`. Implementacja może zapisywać dane do bazy danych, do pliku XML lub wysyłać je przez sieć – kontrakt jest nadal spełniony.
- Dobrze: `PobierzUzytkownikow()` zamiast `WykonajZapytanieSQLDoTabeliUsers()`.

Implementacja interfejsu

Aby zaimplementować interfejs, jego nazwę dodajemy po dwukropku za nazwą klasy. Klasa musi następnie dostarczyć publiczną implementację dla każdego elementu zadeklarowanego w interfejsie. **Sygnatura metody lub właściwości musi dokładnie odpowiadać tej z interfejsu.**

- ConsoleLogger implementuje interfejs ILogger, więc musi zawierać metodę Log.
- Zmienna typu ILogger może wskazywać na dowolny obiekt, który ten interfejs implementuje.

```
interface ILogger
{
    void Log(string message);
}

class ConsoleLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine($"[INFO] {DateTime.Now}: {message}");
    }
}

class Program
{
    static void Main()
    {
        ILogger logger = new ConsoleLogger();
        logger.Log("Program uruchomiony.");
    }
}
```

Implementacja interfejsów

```
interface ILogger
{
    void Log(string message);
}

interface IFileWritable
{
    void SaveToFile(string path);
}

class FileLogger : ILogger, IFileWritable
{
    private string _content = "";

    public void Log(string message)
    {
        _content += $"[LOG] {message}\n";
    }

    public void SaveToFile(string path)
    {
        File.WriteAllText(path, _content);
    }
}
```

```
FileLogger logger = new FileLogger();
logger.Log("Zapis rozpoczęty.");
logger.SaveToFile("log.txt");
```

Klasa może implementować wiele interfejsów.

Na tym przykładzie:

- Klasa FileLogger implementuje dwa interfejsy (ILogger, IFileWritable).
- Każdy z nich wymaga własnych metod — klasa musi dostarczyć **pełną implementację obu kontraktów**.
- Dzięki temu obiekt można traktować jako ILogger, IFileWritable lub FileLogger.

Interfejsy

Zamiast oczekiwać określonego typu (FileLogger, ConsoleLogger itd.), metoda może przyjmować **interfejs** – czyli dowolny obiekt, który spełnia wymagany kontrakt.

Co tu się dzieje?

- ReportGenerator **nie zna konkretnego typu loggera** – interesuje go tylko, że dostarczony obiekt potrafi Log().
- Dzięki temu można łatwo podmienić implementację (ConsoleLogger, FileLogger, DatabaseLogger itp.) **bez zmiany kodu metody**.
- To przykład **polimorfizmu przez interfejs** – różne klasy zachowują się w ten sam sposób z punktu widzenia kodu wywołującego.

Wniosek:

Interfejsy pozwalają pisać kod **elastyczny, testowalny i rozszerzalny**. Dzięki nim komponenty programu komunikują się przez **kontrakty**, a nie przez konkretne typy klas.

```
interface ILogger
{
    void Log(string message);
}

class ConsoleLogger : ILogger
{
    public void Log(string message)
        => Console.WriteLine($"[INFO] {message}");
}

class FileLogger : ILogger
{
    public void Log(string message)
        => File.AppendAllText("log.txt", $"[FILE] {message}\n");
}
```

```
var report = new ReportGenerator();

// Możemy użyć dowolnego loggera
report.Generate(new ConsoleLogger());
report.Generate(new FileLogger());
```

```
class ReportGenerator
{
    public void Generate(ILogger logger)
    {
        logger.Log("Generowanie raportu rozpoczęte...");
        // ...tu logika tworzenia raportu...
        logger.Log("Raport zakończony pomyślnie.");
    }
}
```

Dziedziczenie interfejsów

```
interface IReadable
{
    void Read(string path);
}

interface IWritable
{
    void Write(string path, string content);
}

// Nowy interfejs łączy dwa istniejące
interface IFileHandler : IReadable, IWritable
{
    void Delete(string path);
}

// Klasa implementuje cały rozszerzony kontrakt
class FileManager : IFileHandler
{
    public void Read(string path)
        => Console.WriteLine($"Odczyt z pliku: {path}");

    public void Write(string path, string content)
        => Console.WriteLine($"Zapis do pliku: {path} (treść: {content})");

    public void Delete(string path)
        => Console.WriteLine($"Uswanie pliku: {path}")
}
```

Interfejs może **dziedziczyć po jednym lub wielu innych interfejsach**.

W ten sposób tworzy **rozszerzony kontrakt**, łączący zachowania kilku innych interfejsów.

Warto zauważyć, że:

- IFileHandler **dziedziczy po dwóch interfejsach** (IReadable, IWritable),
- klasa FileManager musi zaimplementować wszystkie metody z IFileHandler, IReadable i IWritable,
- w ten sposób można komponować zachowania bez potrzeby wielokrotnego dziedziczenia klas.

Wniosek:

Dziedziczenie interfejsów pozwala **łączyć mniejsze kontrakty w większe**. Dzięki temu kod staje się **bardziej modułarny i łatwiejszy do rozszerzenia**.

Interfejsy w dużych projektach

Interfejsy to fundament architektury opartej na zależnościach. Pozwalają oddzielić logikę aplikacji od konkretnej implementacji, dzięki czemu kod jest:

- elastyczny,
- testowalny,
- łatwy w rozbudowie.

Co tu się dzieje?

- NotificationManager **nie wie**, jak dokładnie działa wysyłanie e-maila — zna tylko **interfejs** IEmailService.
- Można łatwo podmienić implementację (np. MockEmailService, ApiEmailService) **bez zmiany logiki aplikacji**.
- Tak działa **Dependency Injection (DI)** – jedna z kluczowych praktyk w .NET.

Wniosek:

W dużych projektach interfejsy to **klej spajający moduły aplikacji**. Dzięki nim kod jest **czysty, niezależny i łatwy do utrzymania** — a każdą część można łatwo wymienić.

```
interface IEmailService
{
    void Send(string recipient, string message);
}

class SmtplibEmailService : IEmailService
{
    public void Send(string recipient, string message)
        => Console.WriteLine($"Wysyłam maila do {recipient}: {message}");
}

class NotificationManager
{
    private readonly IEmailService _emailService;

    public NotificationManager(IEmailService emailService)
    {
        _emailService = emailService;
    }

    public void Notify(string user)
        => _emailService.Send(user, "Masz nowe powiadomienie!");
}
```

```
IEmailService service = new SmtplibEmailService();
var manager = new NotificationManager(service);
manager.Notify("user@example.com");
```

Interfejsy a składowe statyczne

Od C# 11 (.NET 7) interfejsy mogą również zawierać statyczne metody i właściwości, należą one jednak do samego interfejsu – **nie do jego instancji**.

Dzięki temu interfejs może zawierać **wspólne zachowanie lub wartości** dostępne bez implementacji.

Uwaga:

Statyczne elementy interfejsu **nie są dziedziczone** przez klasy implementujące. Nie można ich również nadpisywać ani ukrywać. Dostęp do składowych statycznych odbywa się przez **nazwę interfejsu** np. `IShape.DefaultScale`.

```
public interface IShape
{
    // Statyczna właściwość wspólna dla wszystkich figur
    static double DefaultScale => 1.0;

    // Statyczna metoda fabryczna
    static IShape CreateRectangle(double w, double h)
        => new Rectangle(w, h);

    double Area(); // metoda instancyjna
}

public class Rectangle(double width, double height) : IShape
{
    public double Area() => width * height;
}

var rect = IShape.CreateRectangle(4, 5);
Console.WriteLine(rect.Area()); // 20
Console.WriteLine(IShape.DefaultScale); // 1
```

Interfejsy a abstrakcyjne składowe statyczne

```
public interface IAddable<TSelf>
    where TSelf : IAddable<TSelf>
{
    static abstract TSelf operator +(TSelf a, TSelf b);
}

public struct Vector2(double x, double y) : IAddable<Vector2>
{
    public double X { get; } = x;
    public double Y { get; } = y;

    public static Vector2 operator +(Vector2 a, Vector2 b)
        => new(a.X + b.X, a.Y + b.Y);

    public override string ToString() => $"({X}, {Y})";
}

class Program
{
    static void Main()
    {
        var a = new Vector2(2, 3);
        var b = new Vector2(1, 4);
        Console.WriteLine(a + b); // (3, 7)
    }
}
```

Od C# 11 interfejsy również mogą zawierać **statyczne, abstrakcyjne metody i właściwości**, które muszą zostać zaimplementowane w klasach.

Umożliwia to tworzenie **generycznych algorytmów** działających na różnych typach, np. operacji matematycznych (+, *, *Parse()*, *Zero*, itp.).

Dziedziczenie

Dziedziczenie (*inheritance*) to mechanizm programowania obiektowego, który pozwala tworzyć **nowe klasy na podstawie już istniejących**. Klasa pochodna **dziedziczy pola, właściwości i metody** klasy bazowej, może też **rozszerzać lub nadpisywać** jej zachowanie.

W skrócie:

- Klasa bazowa (*base class*) – ogólny typ, np. Animal.
- Klasa pochodna (*derived class*) – typ bardziej szczegółowy, np. Dog, Cat.
- Dziedziczenie opisuje relację „**is a**” (jest):
 - Pies jest zwierzęciem, Kot jest zwierzęciem.

Dziedziczenie

```
class Animal
{
    public void Eat() => Console.WriteLine("Zwierzę je.");
}

class Dog : Animal
{
    public void Bark() => Console.WriteLine("Pies szczeka.");
}

class Program
{
    static void Main()
    {
        Dog dog = new Dog();
        dog.Eat(); // odziedziczone po Animal
        dog.Bark(); // własna metoda
    }
}
```

Co tutaj się dzieje?

- Dog **dziedziczy** wszystko z klasy Animal.
- Dog **może używać metod klasy bazowej**, tak jakby były jego własne.
- Nie trzeba powtarzać kodu – Eat() jest odziedziczone.

Wniosek:

Dziedziczenie pozwala ponownie używać kodu, uporządkować hierarchię klas i uniknąć duplikacji logiki.

Dziedziczenie - zasady

W C# klasa może dziedziczyć tylko po jednej klasie

C# nie wspiera wielokrotnego dziedziczenia klas (w przeciwieństwie do np. C++). Dzięki temu unika się konfliktów i niejednoznaczności.

```
class A { }  
class B : A { } // ✓ OK  
class C : B { } // ✓ OK  
// class D : A, B { } // ✗ Błąd – tylko jedna klasa bazowa
```

Dziedziczenie - zasady

Klasa zawsze pośrednio dziedziczy po object

Nawet jeśli tego nie napiszesz, każda klasa dziedziczy z System.Object.



```
class MyClass { } // automatycznie: class MyClass : object
```

Dziedziczenie - zasady

Elementy oznaczone jako private nie są dziedziczone

Są widoczne tylko wewnątrz klasy bazowej. Klasy pochodne nie mają do nich dostępu (chyba że użyjemy protected).



```
class Base
{
    private int _id;           // niedostępne w klasie pochodnej
    protected string Name { get; set; } // dostępne w klasie pochodnej
}
```

Dziedziczenie - zasady

Konstruktor klasy bazowej zawsze wywoływany jest przed konstruktorem klasy pochodnej

Dzięki temu najpierw inicjalizowany jest ogólny stan obiektu.

```
class Base
{
    public Base() => Console.WriteLine("Base()");
}

class Derived : Base
{
    public Derived() => Console.WriteLine("Derived()");
}

// Output:
// Base()
// Derived()
```

Dziedziczenie - zasady

Słowo kluczowe sealed blokuje dalsze dziedziczenie

Klasy sealed nie mogą być klasami bazowymi.



```
sealed class A { }  
// class B : A { } // ✗ Błąd kompilacji
```

Wniosek:

Dziedziczenie w C# jest proste, jednoznaczne i kontrolowane. Każda klasa ma tylko jednego przodka, ale może dziedziczyć po nim dowolnie głęboko.

Dziedziczenie – słowo kluczowe **base**

Słowo kluczowe **base** jest używane **wewnątrz klasy pochodnej** i działa jak "uchwyt" do jej bezpośredniej klasy bazowej. Pozwala ono na dostęp do tych składowych klasy nadrzędnej, które mogły zostać przesłonięte lub ukryte w klasie potomnej. Ma dwa główne, bardzo ważne zastosowania:

- wywoływanie konstruktora klasy bazowej,
- dostęp do metod klasy bazowej.

Dziedziczenie – słowo kluczowe **base**

base.Speak() wywołuje oryginalną metodę z klasy Animal, a potem klasa Dog dodaje własne zachowanie.

```
//Output:  
Zwierzę wydaje dźwięk.  
Pies szczeka.
```

```
class Animal  
{  
    public virtual void Speak()  
        => Console.WriteLine("Zwierzę wydaje dźwięk.");  
}  
  
class Dog : Animal  
{  
    public override void Speak()  
    {  
        base.Speak(); // wywołanie metody z klasy bazowej  
        Console.WriteLine("Pies szczeka.");  
    }  
}  
  
class Program  
{  
    static void Main()  
    {  
        Dog dog = new Dog();  
        dog.Speak();  
    }  
}
```

Dziedziczenie – słowo kluczowe **base**

```
class Vehicle
{
    public string Brand { get; }

    public Vehicle(string brand)
        => Brand = brand;
}

class Car : Vehicle
{
    public string Model { get; }

    public Car(string brand, string model) : base(brand) // wywołanie konstruktora klasy bazowej
    {
        Model = model;
    }
}
```

base(brand) wywołuje konstruktor Vehicle, przekazując mu parametr brand.
Dzięki temu klasa pochodna **inicjalizuje część odziedziczoną**.

Dziedziczenie – słowo kluczowe **base**

Konstruktor klasy bazowej to metoda specjalna, która jest **zawsze wywoływana przed konstruktorem klasy pochodnej**. Odpowiada za **inicjalizację pól i właściwości odziedziczonych** po klasie bazowej.

```
class Animal
{
    public Animal()
        => Console.WriteLine("Konstruktor Animal");
}

class Dog : Animal
{
    public Dog()
        => Console.WriteLine("Konstruktor Dog");
}

class Program
{
    static void Main() => new Dog();
}
```

Jeśli klasa bazowa ma **konstruktor bezparametrowy**, zostanie on wywołany automatycznie:

```
Konstruktor Animal
Konstruktor Dog
```

Uwaga

Konstruktor klasy bazowej zawsze uruchamia się **pierwszy**.

Dziedziczenie – słowo kluczowe **base**

Jeśli klasa bazowa **nie ma konstruktora domyślnego**, klasa pochodna **musi jawnie wskazać**, który konstruktor ma być wywołany.

`: base(brand)` przekazuje wartość do konstruktora Vehicle.
Bez tego kod nie skompilowałby się, bo Vehicle **nie ma konstruktora bezparametrowego**.

```
class Vehicle
{
    public string Brand { get; }

    public Vehicle(string brand)
        => Brand = brand;
}

class Car : Vehicle
{
    public string Model { get; }

    public Car(string brand, string model)
        : base(brand) // wywołanie konstruktora z klasy bazowej
    {
        Model = model;
    }
}
```

Polimorfizm

- **Polimorfizm** (z greckiego: "wielopostaciowość") – mechanizm, który pozwala traktować obiekty różnych klas pochodnych w jednolity sposób – przez pryzmat ich wspólnej klasy bazowej lub interfejsu. Mimo że operujemy na typie bazowym, w rzeczywistości wywoływane są metody specyficzne dla konkretnych obiektów (klas pochodnych).
- **Przesłanianie** (*shadowing / hiding*) – stworzenie w klasie pochodnej nowej, niezależnej metody lub pola, które ma taką samą nazwę jak składowa w klasie bazowej. Ta nowa składowa "przesłania" (ukrywa) tę z klasy bazowej, ale jej nie zmienia. Składowa bazowa wciąż istnieje i można się do niej dostać, np. przez rzutowanie obiektu na typ bazowy.
- **Nadpisywanie** (*overriding*) – dostarczenie w klasie pochodnej nowej implementacji dla metody, która w klasie bazowej została oznaczona jako virtual lub abstract. Nadpisanie modyfikuje zachowanie istniejącej metody w ramach mechanizmu polimorfizmu.

Polimorfizm

Dziedziczenie pozwala klasie pochodnej **zmieniać lub ukrywać zachowanie** metod klasy bazowej. Do tego służą słowa kluczowe:

- **virtual** – metoda może być nadpisana,
- **override** – metoda nadpisuje metodę w klasie bazowej,
- **new** – metoda ukrywa metodę bazową – przesłania (*shadowing*).

Nadpisywanie

```
class Animal
{
    public virtual void Speak()
        => Console.WriteLine("Zwierzę wydaje dźwięk.");
}

class Dog : Animal
{
    public override void Speak()
        => Console.WriteLine("Pies szczeka.");
}
```

```
Animal a = new Animal();
a.Speak(); // Zwierzę wydaje dźwięk.

Dog d = new Dog();
d.Speak(); // Pies szczeka.

Animal ad = new Dog();
ad.Speak(); // Pies szczeka – dzięki polimorfizmowi
```

- **virtual** – metoda może być nadpisana.
- **override** – metoda nadpisuje zachowanie bazowe.
- **Wywołanie przez typ bazowy (Animal) nadal uruchamia wersję pochodną (Dog).**

Przysłanianie

```
class Animal
{
    public void Speak() => Console.WriteLine("Zwierzę wydaje dźwięk.");
}

class Cat : Animal
{
    public new void Speak() => Console.WriteLine("Kot miauczy.");
}

class Program
{
    static void Main()
    {
        Animal a = new Cat();
        a.Speak(); // ■ Zwierzę wydaje dźwięk. (nie kot!)
    }
}
```

new nie nadpisuje metody – tylko ją **ukrywa**.
Wywołanie przez typ bazowy (Animal) nie korzysta z wersji Cat.

Polimorfizm

Słowo kluczowe	Co robi?	Efekt przy wywołaniu przez typ bazowy
<i>(brak)</i>	metoda zwykła	wywoła metodę bazową
virtual	metoda może być nadpisana	—
override	metoda nadpisuje virtual	wywoła wersję pochodną ✓
new	metoda ukrywa wersję bazową	wywoła wersję bazową ✗

Tabela podsumowująca najważniejsze różnice między nadpisywaniem i przystanianiem.

Rzutowanie obiektów

Rzutowanie to **zmiana typu referencji** obiektu w hierarchii dziedziczenia. Pozwala traktować obiekt klasy pochodnej jako obiekt klasy bazowej (lub odwrotnie).

Rzutowanie w górę (*upcasting*)

```
Dog dog = new Dog();  
Animal animal = dog; // ✅ OK – każdy Dog jest Animal  
animal.Speak();      // Wywoła metodę z Dog (dzięki polimorfizmowi)
```

Rzutowanie **w górę hierarchii** (z Dog → Animal) jest **bezpieczne** i wykonywane **automatycznie** przez kompilator. Nie trzeba nawet używać nawiasów.

Rzutowanie w dół (*downcasting*)

```
Animal animal = new Dog();  
Dog dog = (Dog)animal; // ✅ OK, bo faktycznie pod spodem jest Dog  
  
Animal a2 = new Animal();  
Dog d2 = (Dog)a2;      // ❌ Błąd w czasie działania (InvalidCastException)
```

Rzutowanie **w dół hierarchii** (z Animal → Dog) jest **dozwolone składniowo**, ale **ryzykowne** – jeśli obiekt nie jest naprawdę danego typu, program się wykrzaczy (rzuci wyjątkiem).

Uwaga

Najważniejsze jest to, instancję jakiej klasy tworzymy z użyciem operatora **new**.

Polimorfizm a kolekcje

Lista `List<Animal>` może przechowywać **dowolne obiekty pochodne**.

Ale `List<Pig>` nie może przyjąć `Dog` ani `Animal` – typ generyczny musi być **dokładnie zgodny**.

```
List<Animal> animals = new()
{
    new Dog(),
    new Cat(),
    new Pig()
};

// ✅ Możemy przechowywać różne zwierzęta w jednej liście
foreach (var a in animals)
{
    a.Speak();
}

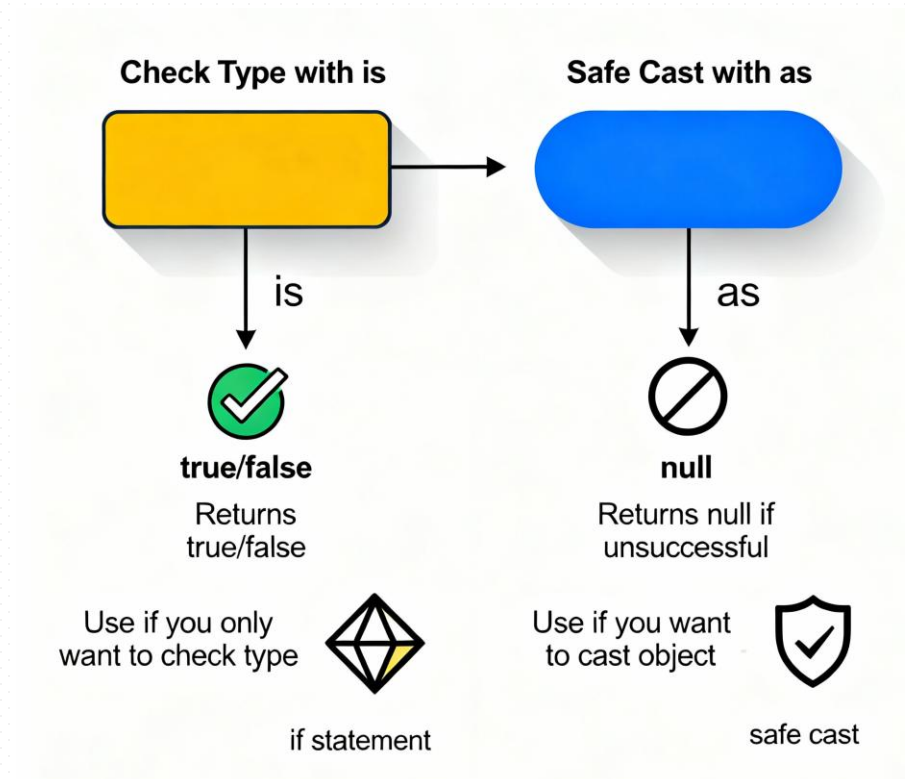
// ❌ Ale nie odwrotnie:
List<Pig> pigs = new();

pigs.Add(new Dog()); // ❌ Błąd kompilacji
pigs = animals;      // ❌ Błąd kompilacji
```

Typy generyczne poznamy na następnej lekcji! ✅💪😄

Bezpieczne rzutowanie – **is** i **as**

Operatory **is** i **as** pozwalają **sprawdzić lub wykonać rzutowanie bez ryzyka błędu** (czyli bez `InvalidCastException`).



Bezpieczne rzutowanie - `is`

```
Animal animal = new Dog();

if (animal is Dog dog)
{
    Console.WriteLine("To jest pies 🐶");
    dog.Speak(); // automatycznie zrzutowany do Dog
}

else if (animal is Cat cat)
{
    Console.WriteLine("To jest kot 🐱");
    cat.Speak();
}

else
{
    Console.WriteLine("Nieznane zwierzę 🐾");
}
```

```
To jest pies 🐶
Pies szczeka.
```

- `is` sprawdza typ i tworzy zmienną danego typu w tym samym wyrażeniu.
- Brak potrzeby pisania `(Dog)animal`.
- Nie generuje wyjątku.

Bezpieczne rzutowanie – as

- **as** próbuje rzutować obiekt do podanego typu, jeśli się nie powiedzie – zwraca null, zamiast rzucać wyjątek.
- Działa tylko z **typami referencyjnymi** (klasy, interfejsy).

```
Animal a = new Cat();  
  
Dog dog = a as Dog; // jeśli się nie uda → dog = null  
  
if (dog == null)  
    Console.WriteLine("To nie jest pies!");  
else  
    dog.Speak();
```

```
To nie jest pies!
```

Wniosek:

is – sprawdza i od razu rzutuje, jeśli się uda

as – rzutuje bezpiecznie, zwraca null przy niepowodzeniu

Klasy abstrakcyjne

Klasa abstrakcyjna to **taka, której nie można utworzyć** (*stworzyć jej instancji*), ale może służyć jako **podstawa dla innych klas**. Zawiera **wspólne cechy i zachowania**, które dziedziczą klasy pochodne.

Cechy klas abstrakcyjnych:

- Nie można utworzyć obiektu klasy abstrakcyjnej (new jest zabronione).
- Mogą zawierać:
 - zwykłe metody z implementacją,
 - właściwości,
 - pola,
 - metody abstrakcyjne.
- Służą jako wspólny **szablon dla grupy klas pochodnych**.

Klasy abstrakcyjne

```
abstract class Animal
{
    public void Eat()
        => Console.WriteLine("Zwierzę je.");
}

class Dog : Animal
{
    public void Bark()
        => Console.WriteLine("Pies szczeka.");
}

class Program
{
    static void Main()
    {
        // Animal a = new Animal(); // ❌ Błąd – klasa abstrakcyjna
        Dog dog = new Dog();         // ✅ OK
        dog.Eat();                    // odziedziczone
        dog.Bark();                   // własne zachowanie
    }
}
```

Zwierzę je.
Pies szczeka.

Kiedy używać klas abstrakcyjnych?

- Gdy chcesz stworzyć **wspólny punkt wyjścia** dla wielu podobnych klas.
- Gdy część zachowań jest **taka sama**, a część **różni się** w zależności od typu pochodnego.
- Gdy chcesz wymusić na klasach pochodnych **posiadanie pewnych metod lub właściwości** (np. przez metody abstrakcyjne).

Wniosek:

Klasa abstrakcyjna to **niedokończony szablon** – definiuje wspólny fundament, ale wymaga uzupełnienia przez klasy pochodne.

Metody abstrakcyjne

Metoda abstrakcyjna to **metoda bez implementacji**, która musi zostać **nadpisana (override)** w każdej klasie dziedziczącej po klasie abstrakcyjnej.

Cechy metod abstrakcyjnych:

- Można je deklarować **tylko w klasach abstrakcyjnych**.
- Nie mają **ciała { }**.
- Klasy pochodne **muszą** je zaimplementować.
- Wymuszają spójność w hierarchii – każda podklasa **musi zachowywać się zgodnie z określonym kontraktem**.

Metody abstrakcyjne

```
abstract class Animal
{
    public abstract void Speak(); // brak implementacji
    public void Eat() => Console.WriteLine("Zwierzę je.");
}

class Dog : Animal
{
    public override void Speak()
        => Console.WriteLine("Pies szczeka.");
}

class Cat : Animal
{
    public override void Speak()
        => Console.WriteLine("Kot miauczy.");
}
```

Wniosek:

Metody abstrakcyjne to **obowiązkowe punkty rozszerzenia** – definiują *co musi istnieć*, ale nie *jak ma działać*. Dzięki nim klasy pochodne zachowują wspólny interfejs, a jednocześnie mogą działać **po swojemu**.

Co tu się dzieje?

- Klasa `Animal` określa wspólne zachowanie (`Eat()`) oraz definiuje metodę abstrakcyjną `Speak()`, którą trzeba uzupełnić.
- Klasy `Dog` i `Cat` same decydują, jak zwierzę mówi.
- Dzięki temu każda podklasa zachowuje się inaczej – ale cały kod pracuje z typem `Animal`.

```
List<Animal> animals = new() { new Dog(), new Cat() };

foreach (var a in animals)
{
    a.Eat();
    a.Speak();
}
```

Zwierzę je.
Pies szczeka.
Zwierzę je.
Kot miauczy.

Klasy zapieczętowane – sealed

Klasa oznaczona słowem kluczowym **sealed** to klasa zapieczętowana, po której **nie można dziedziczyć**. Oznacza to, że jest to **ostateczna wersja** klasy – nie da się jej rozszerzyć ani nadpisać.

```
sealed class PaymentProcessor
{
    public void Process()
        => Console.WriteLine("Przetwarzanie płatności...");
}

// ✗ Błąd – nie można dziedziczyć po klasie sealed
// class AdvancedProcessor : PaymentProcessor { }

class Program
{
    static void Main()
    {
        PaymentProcessor processor = new PaymentProcessor();
        processor.Process();
    }
}
```


Przetwarzanie płatności...

Klasa działa normalnie, ale nie może mieć klas pochodnych.

Metody nadpisane i zapieczętowane

Słowo **sealed** można też połączyć z **override**, aby **zablokować dalsze nadpisywanie metody** w klasach niższego poziomu.

```
class Animal
{
    public virtual void Speak() => Console.WriteLine("Zwierzę wydaje dźwięk.");
}

class Dog : Animal
{
    public sealed override void Speak() => Console.WriteLine("Pies szczeka.");
}

// ✗ Klasa Husky nie może już zmienić metody Speak()
class Husky : Dog
{
    // public override void Speak() => Console.WriteLine("Husky wyje."); // Błąd
}
```

sealed override – metoda jest dziedziczona, ale *nie można już jej ponownie nadpisać*.

Wniosek:

sealed pozwala **ograniczyć dziedziczenie**, gdy klasa lub metoda ma być **ostateczna i niezmiennalna**. Chroni przed błędami, gdy dalsze rozszerzanie nie ma sensu lub byłoby niebezpieczne.

Podstawowe metody obiektów

Każda klasa w C# dziedziczy po **System.Object**. Dlatego wszystkie obiekty mają pewien wspólny zestaw metod.

Metoda	Opis
Equals(object obj)	Sprawdza, czy dwa obiekty są równe (domyślnie porównuje referencje).
GetHashCode()	Zwraca liczbę całkowitą (<i>hash</i>), reprezentującą obiekt – używana np. w słownikach (<i>Dictionary</i>).
ToString()	Zwraca tekstową reprezentację obiektu. Domyślnie – nazwę klasy.
GetType()	Zwraca typ (<i>metadane</i>) obiektu w czasie działania.

Metody z powyższej tabelki (z wyłączeniem `GetType()`), **występują w klasie `Object` jako `virtual`** – oznacza to, że możemy je nadpisać.

Podstawowe metody obiektów

```
class Car
{
    public string Brand { get; set; }

    public override string ToString()
        => $"Samochód marki {Brand}";
}

class Program
{
    static void Main()
    {
        Car car = new Car { Brand = "Honda" };

        Console.WriteLine(car.ToString());    // Samochód marki Honda
        Console.WriteLine(car.GetType());     // Namespace.Car
        Console.WriteLine(car.Equals(car));   // True
        Console.WriteLine(car.GetHashCode()); // np. 12345678
    }
}
```

Przykład działania podstawowych metod.

Porównanie wartości – Equals()

Domyślnie Equals() porównuje **referencje**, ale można ją **nadpisać**, aby porównywać **zawartość** obiektów:

```
class Point
{
    public int X { get; set; }

    public int Y { get; set; }

    public override bool Equals(object? obj)
    {
        if (obj is Point p)
            return X == p.X && Y == p.Y;

        return false;
    }

    public override int GetHashCode()
        => GetHashCode.Combine(X, Y);
}
```



```
var p1 = new Point() { X = 1, Y = 2 };
var p2 = new Point() { X = 1, Y = 2 };

if (p1.Equals(p2))
{
    Console.WriteLine("Punkty są identyczne!");
}

//Wynik: Punkty są identyczne!
```

Teraz dwa punkty, z tymi samymi współrzędnymi będą uznane za równe.

Metoda ToString()

Metoda ToString() zwraca **tekstową reprezentację obiektu**. Domyślnie zwraca nazwę typu, ale można ją **nadpisać**, aby prezentowała dane w bardziej czytelny sposób.

```
class Car
{
    public string Brand { get; set; }
    public string Model { get; set; }
}

class Program
{
    static void Main()
    {
        Car car = new Car { Brand = "VW", Model = "Passat" };
        Console.WriteLine(car.ToString());
    }
}

//Wynik: Namespace.Car
```

Domyślnie ToString() zwraca pełną nazwę typu (namespace + nazwa klasy).

```
class Car
{
    public string Brand { get; set; }
    public string Model { get; set; }
    public int HorsePower { get; set; }

    public override string ToString()
        => $"{Brand} {Model} ({HorsePower} KM)";
}

class Program
{
    static void Main()
    {
        Car car = new Car { Brand = "VW", Model = "Passat", HorsePower = 110 };
        Console.WriteLine(car); // automatycznie wywoła ToString()
    }
}

//Wynik: VW Passat (110 KM)
```

ToString() może być wywoływane **jawnie** lub **automatycznie**, np. przy Console.WriteLine() lub interpolacji łańcuchów (\$"Auto: {car}").

Metoda `Equals()` i interfejs `IEquatable<T>`

Interfejs `IEquatable<T>` pozwala zdefiniować **wydatny i typowany sposób** porównywania obiektów. Używa się go zamiast ogólnego `Equals(object)` — dzięki temu unikamy rzutowania i zwiększamy czytelność.

Dlaczego samo `Equals(object)` nie wystarcza?

`Equals(object)` przyjmuje argument typu `object`, więc przy porównaniu dwóch obiektów trzeba wykonać **rzutowanie**. Sposób ten działa, ale jest **mniej wydajny i bardziej ogólny**. Dlatego stosujemy `IEquatable<T>`, które umożliwia porównanie obiektów bez rzutowania.

```
public override bool Equals(object? obj)
{
    if (obj is Point p)
        return X == p.X && Y == p.Y;

    return false;
}
```

Metoda Equals() i interfejs IEquatable<T>

```
class Point : IEquatable<Point>
{
    public int X { get; set; }
    public int Y { get; set; }

    public bool Equals(Point? other)
    {
        if (other == null)
            return false;

        return X == other.X && Y == other.Y;
    }

    public override bool Equals(object? obj)
        => Equals(obj as Point);

    public override int GetHashCode()
        => HashCode.Combine(X, Y);

    public override string ToString()
        => $"({X}, {Y})";
}
```

```
var p1 = new Point { X = 3, Y = 5 };
var p2 = new Point { X = 3, Y = 5 };
var p3 = new Point { X = 4, Y = 5 };

Console.WriteLine(p1.Equals(p2)); // ✓ True
Console.WriteLine(p1.Equals(p3)); // ✗ False
```

Dzięki **IEquatable<Point>** porównywanie jest:

- **szybsze** (bez rzutowania do object),
- **typowane** (nie można przypadkiem porównać różnych typów),
- **czytelne** w użyciu.

IEquatable<T> to zalecany sposób porównywania obiektów. Zapewnia spójność między Equals() i GetHashCode(), co jest kluczowe przy pracy z kolekcjami (HashSet, Dictionary, Distinct() itd.).

Identyfikator obiektu – GetHashCode()

Metoda **GetHashCode()** zwraca liczbę całkowitą (**hash**) reprezentującą dany obiekt. Używana jest m.in. przez kolekcje (Dictionary, HashSet, Lookup) do szybkiego wyszukiwania elementów.

Każdy obiekt w C# ma wartość **hash**, dla poprawnego działania kolekcji obowiązują zasada:

Jeżeli **a.Equals(b)** jest **true**, to **a.GetHashCode() == b.GetHashCode()** musi być **true**.

Odwrotność nie jest wymagana

Identyfikator obiektu – GetHashCode()

Domyślna implementacja **GetHashCode()** w **object** zwraca **hash referencji** – czyli różne wartości dla różnych obiektów, nawet jeśli mają te same dane.

```
var s1 = "Hello";  
var s2 = "Hello";  
Console.WriteLine(s1.Equals(s2)); // True  
Console.WriteLine(s1.GetHashCode()); // np. 69603056  
Console.WriteLine(s2.GetHashCode()); // np. 69603056 (string ma nadpisany hash!)
```

Klasa string nadpisuje GetHashCode(), dlatego dwie równe wartości mają ten sam hash.

Implementacja GetHashCode()

```
class Point : IEquatable<Point>
{
    public int X { get; set; }
    public int Y { get; set; }

    public bool Equals(Point? other)
        => other is not null && X == other.X && Y == other.Y;

    public override int GetHashCode()
        => GetHashCode.Combine(X, Y);
}
```

HashCode.Combine() (C# 8+) automatycznie generuje dobry, unikalny hash z wielu pól.

Znaczenie GetHashCode()

```
HashSet<Point> points = new()  
{  
    new Point { X = 1, Y = 2 },  
    new Point { X = 1, Y = 2 } // ten sam punkt  
};  
  
Console.WriteLine(points.Count); // powinno być 1
```

*Jeśli **GetHashCode()** nie byłby zgodny z **Equals()**, kolekcja mogłaby uznać oba obiekty za różne, więc wynikiem byłoby 2.*

Wniosek:

GetHashCode() musi być spójny z **Equals()**. Dzięki temu kolekcje działają poprawnie i wydajnie. Używaj **HashCode.Combine()** lub bibliotek hashujących – nigdy nie zwracaj $X + Y$.

Manualna implementacja GetHashCode()

Przed C# 8.0, było trzeba ręcznie implementować metodę **GetHashCode()** – nie było **HashCode.Combine()** – jest ona niedostępna chociażby w **.NET Standard 2.0**. W takim przypadku, generuje się hashe ręcznie, korzystając z bloku **unchecked**.

```
public override int GetHashCode()  
{  
    unchecked // ignoruje przepełnienia arytmetyczne  
    {  
        int hash = 17;  
        hash = hash * 23 + X.GetHashCode();  
        hash = hash * 23 + Y.GetHashCode();  
        return hash;  
    }  
}
```

Uwaga!

Blok **unchecked** powoduje, że przepełnienie np. typu **int** nie prowadzi do wyrzucenia wyjątku **System.OverflowException**. Zamiast tego, po przekroczeniu maksymalnej wartości (**Int32.MaxValue**), kolejna wartość staje się minimalną ujemną wartością (**Int32.MinValue**).

Sprawdzanie typu obiektu – GetType()

GetType() to metoda klasy **object**, która zwraca rzeczywisty typ obiektu (instancję klasy **System.Type**). Dzięki niej można w czasie działania programu sprawdzić, z jakiego typu naprawdę pochodzi dany obiekt.



```
Animal animal = new Dog();

Console.WriteLine(animal.GetType()); //Namespace.Dog
Console.WriteLine(animal.GetType().Name); //Dog
Console.WriteLine(animal.GetType().BaseType?.Name); //Animal
```

GetType() zawsze zwraca faktyczny typ obiektu, nawet jeśli zmienna ma typ bazowy (*Animal*).

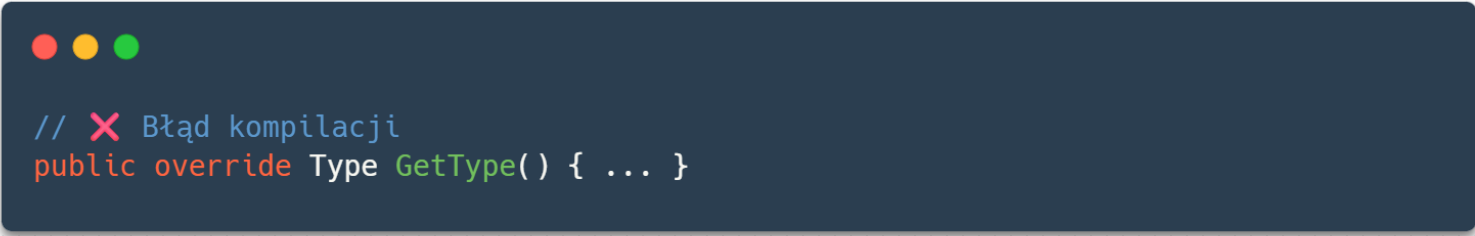
Porównywanie typów



```
Animal a = new Dog();  
Animal b = new Cat();  
  
if (a.GetType() == b.GetType())  
    Console.WriteLine("Ten sam typ");  
else  
    Console.WriteLine("Różne typy");  
  
//Wynik: Różne typy
```

Nadpisywanie `GetType()`

Metoda `GetType()` jest oznaczona jako **sealed** w klasie **object**, co oznacza, że nie można jej nadpisać ani zmienić jej działania.



```
// ✖ Błąd kompilacji  
public override Type GetType() { ... }
```

System .NET musi mieć gwarancję, że `GetType()` zawsze działa w ten sam sposób – niezależnie od klasy lub dziedziczenia.

Wniosek:

Metoda `GetType()` pozwala uzyskać rzeczywisty typ obiektu w czasie działania. Nie można jej nadpisać, dzięki czemu jest bezpieczna i niezawodna. Często używana przy refleksji, debugowaniu, logowaniu i diagnostyce.

Struktury

struct (*struktura*) to **typ wartościowy**, który przechowuje dane **bezpośrednio** w zmiennej, a nie przez referencję (jak obiekt klasy). Struktury są idealne dla **małych, niezmiennych obiektów danych**.

Kluczowe cechy:

- Tworzone za pomocą słowa kluczowego **struct**.
- Są typami wartościowymi – kopiują się przez wartość, nie przez referencję.
- Nie mogą dziedziczyć po innej strukturze ani klasie (**struct** : **class** ✗).
- Mogą implementować interfejsy.
- Nie mogą mieć konstruktora bezparametrowego (*kompilator generuje go automatycznie*).
- Często używane do reprezentowania prostych danych: współrzędnych, kolorów, punktów, zakresów itd.

Struktury

```
struct Point
{
    public int X { get; set; }
    public int Y { get; set; }

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }

    public override string ToString() => $"({X}, {Y})";
}

class Program
{
    static void Main()
    {
        Point p1 = new Point(3, 5);
        Point p2 = p1; // kopia, nie referencja

        p2.X = 10;

        Console.WriteLine(p1); // (3, 5)
        Console.WriteLine(p2); // (10, 5)
    }
}
```

```
//Wynik:
(3, 5)
(10, 5)
```

Struktury (**struct**) są **lekkie i szybkie**, ale przechowują dane **na stosie**, więc najlepiej nadają się do małych obiektów. W przeciwieństwie do klas, są kopiowane **przez wartość**, nie przez referencję.

Do przykładu:

p1 i p2 to dwie niezależne kopie – **zmiana jednej nie wpływa na drugą!**

Konstruktor struktur

Struktura zawsze posiada konstruktor bezparametrowy

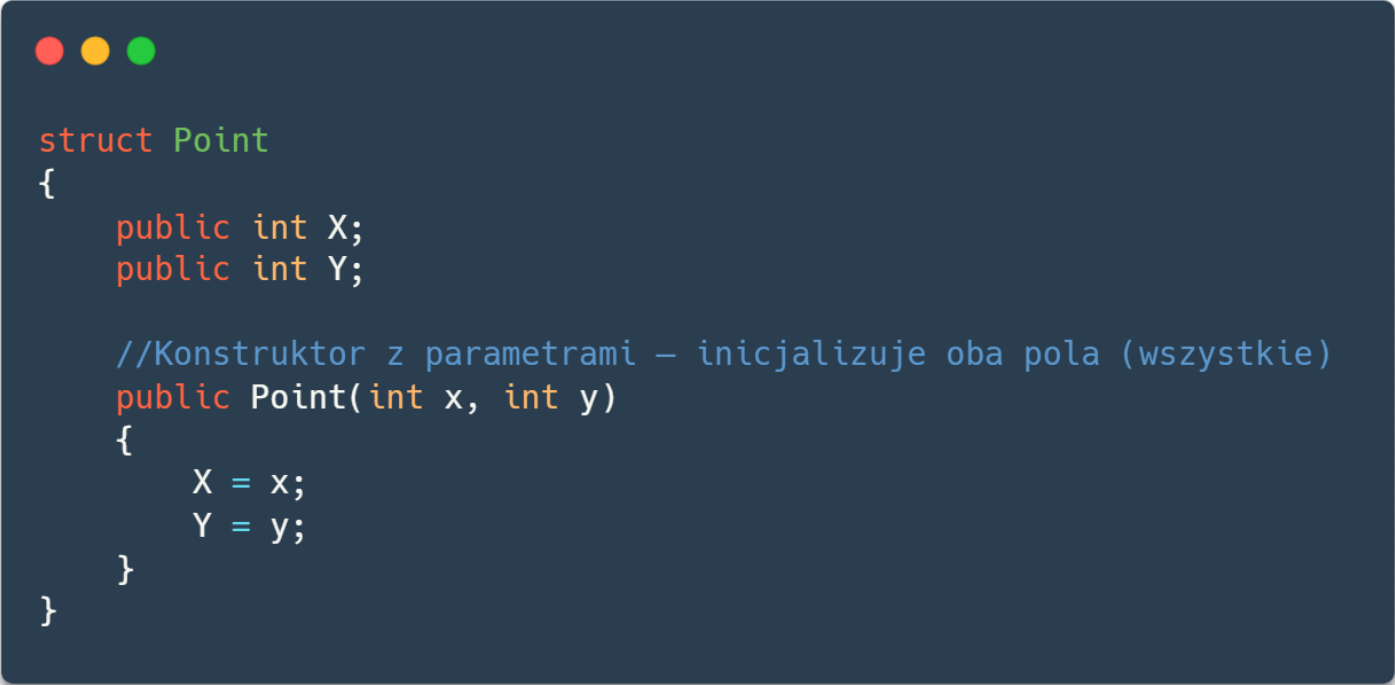
- Jest automatycznie generowany przez kompilator.
- Inicjuje wszystkie pola wartościami domyślnymi (0, false, null).
- Nie można go **nadpisać** ani **usunąć**.



```
Point p = new Point(); // X = 0, Y = 0
```

Konstruktory struktur

Można zdefiniować własny konstruktor z parametrami, ale musi on inicjalizować wszystkie pola struktury.



```
struct Point
{
    public int X;
    public int Y;

    //Konstruktor z parametrami – inicjalizuje oba pola (wszystkie)
    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

Jeśli nie zainicjujesz wszystkich pól, wystąpi błąd kompilacji!

Konstruktory struktur

Nie można tworzyć konstruktora bezparametrowego w strukturze – jest on tworzony przez kompilator.

```
struct BadPoint
{
    public int X { get; set; }
    public int Y { get; set; }

    // ✗ Błąd – struktury nie mogą mieć własnego konstruktora bez parametrów
    public BadPoint() { }
}
```

Wniosek:

Struktury są proste, ale mniej elastyczne niż klasy. Kompilator zawsze zapewnia konstruktor domyślny, a jeśli definiujesz własny – musisz zainicjować wszystkie pola.

Kiedy używać klasy, a kiedy struktury?

Używaj klasy, gdy:

- Obiekt ma **złożone zachowanie lub logikę**.
- Obiekt **powinien być przekazywany przez referencję** (*zmiany mają być widoczne w wielu miejscach*).
- Potrzebne jest **dziedziczenie** lub **polimorfizm**.
- Potrzebne są **finalizatory** (*destructor*) lub **interfejs IDisposable**.
- Obiekt jest **duży** (*np. zawiera kolekcje, zagnieżdżone obiekty itp.*).

Przykłady: User, Order, Car, DatabaseConnection, HttpClient, Stream

Używaj struktury, gdy:

- Obiekt reprezentuje **niewielką porcję danych**, np. współrzędne, kolor, wektor, rozmiar.
- Obiekt jest **niemutowalny** (*immutable*) – jego stan nie powinien się zmieniać po utworzeniu.
- Tworzenie kopii ma być **szybkie i tanie**.
- Nie potrzebujesz dziedziczenia ani rozbudowanej hierarchii typów.

Przykłady: Point, Rectangle, DateTime, Guid, TimeSpan

Kiedy używać klasy, a kiedy struktury?

```
//Struktura – kopia danych
Point a = new Point(3, 5);
Point b = a; //kopia
b.X = 10;
Console.WriteLine(a.X); // 3

//Klasa – referencja do tego samego obiektu
Car c1 = new Car("Fiat");
Car c2 = c1;
c2.Brand = "BMW";
Console.WriteLine(c1.Brand); //BMW
```

Wniosek:

- **Struktura** (struct) – dane są zawsze kopiowane.
- **Klasa** (class) – przekazywana jest zawsze referencja.

Rekordy

Rekord (record) to specjalny typ w C#, który służy do **reprezentowania danych** w sposób **niezmienny** (*immutable*). Rekordy automatycznie implementują `Equals()`, `GetHashCode()` i `ToString()` – dzięki czemu są idealne do przechowywania **wartości logicznie równych**, a nie identycznych w pamięci.

Dlaczego powstały rekordy?

- Klasy w C# porównywane są **przez referencję**, a nie przez wartość.
- Rekordy rozwiązują ten problem, wprowadzając **porównywanie przez wartość**.
- Dodatkowo automatyzują generowanie metod i właściwości.

Rekordy

```
record Person(string Name, int Age);

class Program
{
    static void Main()
    {
        var p1 = new Person("Jan", 20);
        var p2 = new Person("Jan", 20);

        Console.WriteLine(p1 == p2); // ✓ True – porównanie po wartościach
        Console.WriteLine(p1.Equals(p2)); // ✓ True
        Console.WriteLine(p1); // Person { Name = Jan, Age = 20 }
    }
}
```

Przykład obrazujący porównywanie dwóch różnych rekordów, o tych samych właściwościach.
Zwróć uwagę, jak łatwe jest definiowanie rekordów.

record sam generuje:

- `Equals()` i `GetHashCode()` – oparte na polach,
- `ToString()` – w formacie `{ Property = Value }`,
- operator `==` – porównujący wartości.

Definiowanie rekordów

Rekordy można definiować w trzech głównych formach:

- **Pozycyjnej** – najkrótsza.
- **Klasycznej** – pełna definicja właściwości.
- **Z dziedziczeniem** lub dodatkowymi członkami.

Rekord pozycyjny

Rekord pozycyjny (*skrótowa składnia*) to najprostsza i często stosowana forma – wszystkie właściwości są automatycznie { **get; init;** }.

```
public class Program
{
    public record Person(string Name, int Age);

    public static void Main()
    {
        var p = new Person("Jan", 20);
        Console.WriteLine(p.Name); // Jan
        Console.WriteLine(p); //Person { Name = Jan, Age = 20 }
    }
}
```

W tym przypadku, automatycznie tworzone są:

- Konstruktor z parametrami (Name, Age).
- Automatyczne właściwości tylko do odczytu { **get; init;** }
- Metody Equals(), GetHashCode(), ToString().

Rekord z pełną definicją wartości

Dłuższa, ale bardziej elastyczna forma – można dodać np. walidację lub metody.

```
public record Product
{
    public string Name { get; init; } = "";
    public decimal Price { get; init; }

    public string Display() => $"{Name} - {Price} zł";
}

var prod = new Product { Name = "Laptop", Price = 4999 };
Console.WriteLine(prod.Display());

//Output: Laptop - 4999 zł
```

*Własne metody i logika – ale nadal niezmiennosc (**init** zamiast **set**).*

Rekord z dziedziczeniem

Rekordy mogą dziedziczyć po innych rekordach, ale **tylko**, jeśli obie deklaracje są typu **record**.

```
public record Vehicle(string Brand);  
public record Car(string Brand, string Model) : Vehicle(Brand);
```

Rekord Car dziedziczy właściwość Brand i dodaje Model.

Wniosek:

Rekordy mogą być krótkie i zwarte (pozycyjne) lub rozbudowane jak klasy. **W obu przypadkach oferują porównywanie przez wartość, niezmienność** i automatycznie generowane metody pomocnicze.

Rekordy z wymaganymi właściwościami

Rekordy mogą posiadać właściwości, których **wartość jest wymagana podczas inicjalizacji** – określa to słowo kluczowe **required**. Jeżeli nie podamy wszystkich wymaganych wartości, kod się nie skompiluje.

```
public record Person
{
    public string Name { get; init; }
    public required int Age { get; init; }
}

//Jeżeli spróbujemy utworzyć rekord bez wymaganej właściwości...
var settings = new FigureSettings
{
    Name = "Jan";
}

//✗ Błąd kompilacji - nie podaliśmy wymaganej wartości dla Age
```

Zachowanie właściwości w rekordach

Rekordy w C# zostały stworzone z myślą o **niemutowalności**. Sposób deklaracji właściwości bezpośrednio wpływa na ich użycie.



```
public record Person
{
    public string Name { get; init; }
}

var p1 = new Person { Name = "Jan" };
var p2 = p1 with { Name = "Karol" }; // ✓
```

Rekord który jest zawsze niezmienny



```
public record Person
{
    public string Name { get; set; }
}

var p = new Person { Name = "Jan" };
p.Name = "Karol"; // ✗ mutuje obiekt
```

Przykład kodu, który jest anty-patternem

Rekordy – Equals() i GetHashCode()

Rekordy w C# generują automatycznie kilka przydatnych metod, które czynią je **lekkimi, czytelnymi i praktycznymi**. Posiadają automatyczną implementację metod `Equals()` i `GetHashCode()` – opartą na wszystkich jego właściwościach. Dzięki temu dwa rekordy o identycznych danych, są logicznie równe.

```
record Person(string Name, int Age);

var p1 = new Person("Jan", 20);
var p2 = new Person("Jan", 20);
var p3 = new Person("Ola", 22);

Console.WriteLine(p1.Equals(p2)); // ✓ True
Console.WriteLine(p1 == p3);      // ✗ False
Console.WriteLine(p1.GetHashCode() == p2.GetHashCode()); // ✓ True
```

Przykład porównywania rekordów.

Rekordy – ToString()

Rekordy mają czytelną, automatyczną reprezentację tekstową:

```
public class Program
{
    public record Person(string Name, int Age);

    public static void Main()
    {
        var p = new Person("Jan", 20);
        Console.WriteLine(p.Name); // Jan
        Console.WriteLine(p); //Person { Name = Jan, Age = 20 }
    }
}
```

Nie trzeba (*ale można*) nadpisywać metody ToString() – kompilator generuje ją automatycznie.

Rozkładanie rekordów – Deconstruct()

Każdy rekord pozycyjny automatycznie dostaje metodę Deconstruct(), która pozwala rozłożyć obiekt na jego elementy:

```
var (name, age) = p1;  
  
Console.WriteLine(name); // Jan  
Console.WriteLine(age);  // 20
```

Pod spodem kryje się to

```
public void Deconstruct(out string Name, out int Age)  
{  
    Name = this.Name;  
    Age = this.Age;  
}
```

Metoda Deconstruct() jest generowana automatycznie

Uwaga!

Metoda Deconstruct() jest dostępna tylko w przypadku rekordów pozycyjnych!

Kopiowanie rekordu

Słowo kluczowe **with** umożliwia **utworzenie kopii istniejącego rekordu** z wprowadzonymi zmianami w wybranych właściwościach. Oryginalny obiekt pozostaje **niezmienny** (*immutable*).

Dlaczego **with** działa tylko z rekordami?

- Rekordy są immutable, więc operator **with** gwarantuje bezpieczne tworzenie kopii (*brak ryzyka przypadkowej zmiany oryginału*).

```
record Person(string Name, int Age);

class Program
{
    static void Main()
    {
        var p1 = new Person("Jan", 20);

        // Tworzymy kopię z jedną zmienioną właściwością
        var p2 = p1 with { Age = 21 };

        Console.WriteLine(p1); // Person { Name = Jan, Age = 20 }
        Console.WriteLine(p2); // Person { Name = Jan, Age = 21 }
    }
}
```

Obiekt *p2* to nowa kopia – *p1* nie został zmieniony.
Wszystkie właściwości są kopiowane automatycznie, poza tymi, które nadpiszemy

Rekordy jako typy referencyjne i wartościowe

Rekordy mogą być tworzone jako **typy referencyjne** lub **wartościowe**. Dzięki temu można wybrać, czy chcemy zachowanie jak **klasa**, czy jak **struktura** – przy zachowaniu wszystkich zalet rekordów (*porównywanie po wartościach, with, Deconstruct()*).

```
//record class = record (domyślnie rekordy są typami referencyjnymi)
public record class Person(string Name, int Age);
```

Przechowywane na stercie, obsługują dziedziczenie – jak klasy.

```
//Możemy również sprawić, aby rekord był typem wartościowym
public record struct Point(int X, int Y);
```

Przechowywane na stosie, nie obsługują dziedziczenia – jak struktury.

Oba typy rekordów łączy to, że:

- Są porównywane po wartościach.
- Są niemutowalne.

Typy anonimowe

Typ anonimowy to **tymczasowy obiekt**, który nie ma nazwy, a jego pola definiowane są **bezpośrednio w miejscu tworzenia**. Kompilator automatycznie tworzy dla niego **klasę tylko do odczytu** z właściwościami `get`.



```
var person = new { Name = "Jan", Age = 20 };  
  
Console.WriteLine(person.Name); // Jan  
Console.WriteLine(person.Age); // 20  
Console.WriteLine(person);      // { Name = Jan, Age = 20 }
```

Kompilator tworzy klasę
o właściwościach



```
public string Name { get; }  
public int Age { get; }
```

Uwaga!

Typy anonimowe są niemutowalne!

Porównanie typów anonimowych

Typy anonimowe są porównywane po wartościach, nie po referencji – jeżeli mają takie same właściwości i typy.



```
var a = new { X = 1, Y = 2 };  
var b = new { X = 1, Y = 2 };
```

```
Console.WriteLine(a.Equals(b)); //  True
```

Ograniczenia:

- Nie można ich zwracać z metod (bez dynamic lub object).
- Nie można ich użyć poza miejscem, gdzie zostały utworzone.
- Są zawsze niemutowalne (wszystkie właściwości tylko do odczytu).
- Ich nazwy są generowane przez kompilator i niewidoczne w kodzie.

Uwaga!

Typy anonimowe są używane głównie w LINQ, testach, prostym prototypowaniu. Nie powinny być nadużywane.

Metoda Main()

Metoda Main() to **punkt wejścia** programu w C#. To właśnie od niej rozpoczyna się wykonywanie aplikacji. Zazwyczaj znajduje się w klasie Program.cs.

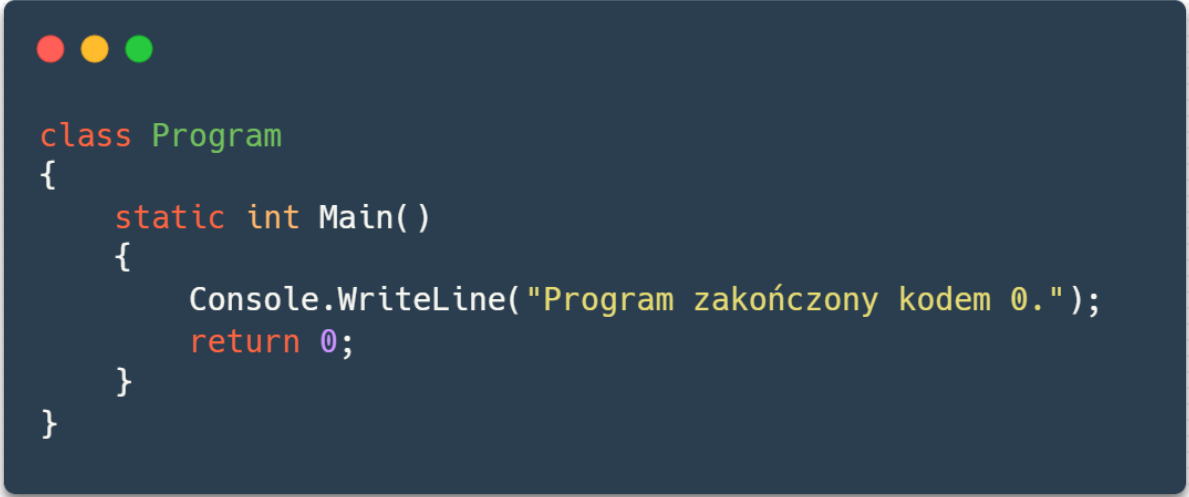


```
class Program
{
    static void Main()
    {
        Console.WriteLine("Hello, World!");
    }
}
```

Należy zapamiętać, że Main() zawsze musi być metodą statyczną oraz że nazwa klasy (Program) nie ma znaczenia, w przypadku braku tej klasy, kompilator będzie poszukiwał Main() w pozostałych klasach.

Metoda Main() – typ zwracany

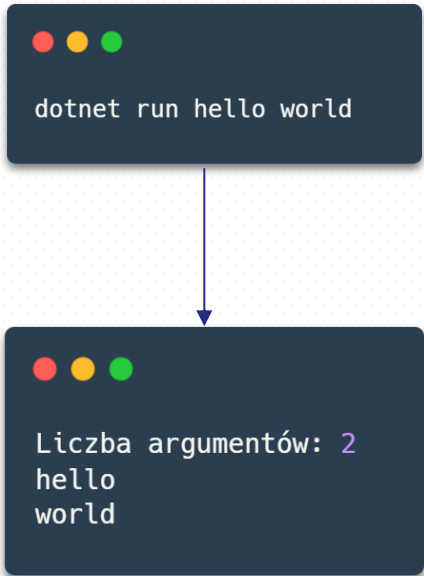
Metoda Main() posiada również swoją **wariację z typem zwracanym int**. Zwracana liczba oznacza **kod zakończenia programu** (0 = sukces, 1 = błąd). Jest to przydatne w szczególności w aplikacjach konsolowych oraz aplikacjach wykonywanych przez inne procesy czy też skrypty – np. CI/CD.



```
class Program
{
    static int Main()
    {
        Console.WriteLine("Program zakończony kodem 0.");
        return 0;
    }
}
```

Metoda `Main()` – sparametryzowana

Metoda `Main()` może przyjmować argumenty, które są **przekazywane przy uruchamianiu programu** (np. przez *terminal*). Jako argumenty **może przyjmować wyłącznie tablicę stringów** – kierując się tym, że wszystko co wpisujemy w konsoli jest tekstem.



```
dotnet run hello world
```

```
Liczba argumentów: 2  
hello  
world
```

```
class Program  
{  
    static void Main(string[] args)  
    {  
        Console.WriteLine($"Liczba argumentów: {args.Length}");  
        foreach (var arg in args)  
            Console.WriteLine(arg);  
    }  
}
```

Instrukcje najwyższego poziomu

Od C# 9 możesz **pomiąć klasę Program i metodę Main**, jeśli w pliku znajduje się tylko jeden blok kodu najwyższego poziomu.



```
Console.WriteLine("Hello, World!");
```

- Kompilator i tak wygeneruje klasę Program w tle.
- Przydatne w małych skryptach.
- Problemатyczne jeżeli w danym pliku mamy zamiar umieścić kilka metod.

Destruktory

Finalizator (zwany potocznie *destruktor*) to specjalna metoda, która jest wywoływana **automatycznie przez Garbage Collector (GC)** tuż przed usunięciem obiektu z pamięci.

Zasady:

- Destruktor ma nazwę `~NazwaKlasy()`.
- Nie przyjmuje parametrów i nie zwraca wartości.
- Nie można go wywołać ręcznie.
- W danej klasie może być **tylko jeden** destruktory.

```
class FileHandler
{
    ~FileHandler()
    {
        Console.WriteLine("Finalizator: zwalniam zasoby pliku...");
    }
}
```

Destruktory

Jak działają destruktory?

- Destruktor nie jest wywoływany natychmiast po utracie referencji.
- Uruchamia go **Garbage Collector (GC)** w nieokreślonym momencie.
- Nie masz gwarancji, **kiedy dokładnie** obiekt zostanie usunięty.

Destruktory **nie nadają się do zwalniania zasobów zewnętrznych** (np. plików, połączeń sieciowych, uchwytów systemowych).

Kiedy przydają się destruktory?

- Tylko w sytuacjach awaryjnych – np. gdy programista zapomni wywołać **Dispose()**.
- Często są używane razem z **IDisposable**, aby mieć podwójną warstwę bezpieczeństwa.

Interfejs **IDisposable**

Interfejs **IDisposable** służy do **ręcznego zwalniania zasobów** niezarządzanych, takich jak pliki, połączenia sieciowe, uchwyty systemowe czy pamięć zewnętrzna. Udostępnia metodę **Dispose()**, którą wywołuje programista lub słowo kluczowe **using**.

```
class FileHandler : IDisposable
{
    private FileStream? _file;

    public FileHandler(string path)
    {
        _file = File.Open(path, FileMode.OpenOrCreate);
    }

    public void Dispose()
    {
        Console.WriteLine("Zwalniam zasoby pliku...");
        _file?.Dispose();
    }
}
```

Po wywołaniu `Dispose()` zasoby zewnętrzne są **zwalniane natychmiast**, bez czekania na Garbage Collector.

Słowo kluczowe `using` oraz `IDisposable`

Słowo kluczowe `using` w kombinacji z obiektem implementującym interfejs `IDisposable`, automatycznie wywołuje metodę `Dispose()` na danym obiekcie po zakończeniu bloku, **nawet jeżeli wewnątrz wystąpi wyjątek**. Słowo kluczowe `using` może występować w dwóch zapisach:

```
using (var handler = new FileHandler("data.txt"))
{
    // operacje
}
```

Metoda `Dispose()` jest wywoływana przy wyjściu z bloku `using`.

```
using var handler = new FileHandler("data.txt");

// ...operacje na pliku...
Console.WriteLine("Zakończono pracę.");
```

Metoda `Dispose()` jest wywoływana przy wyjściu z bloku, w którym znajduje się `using`.